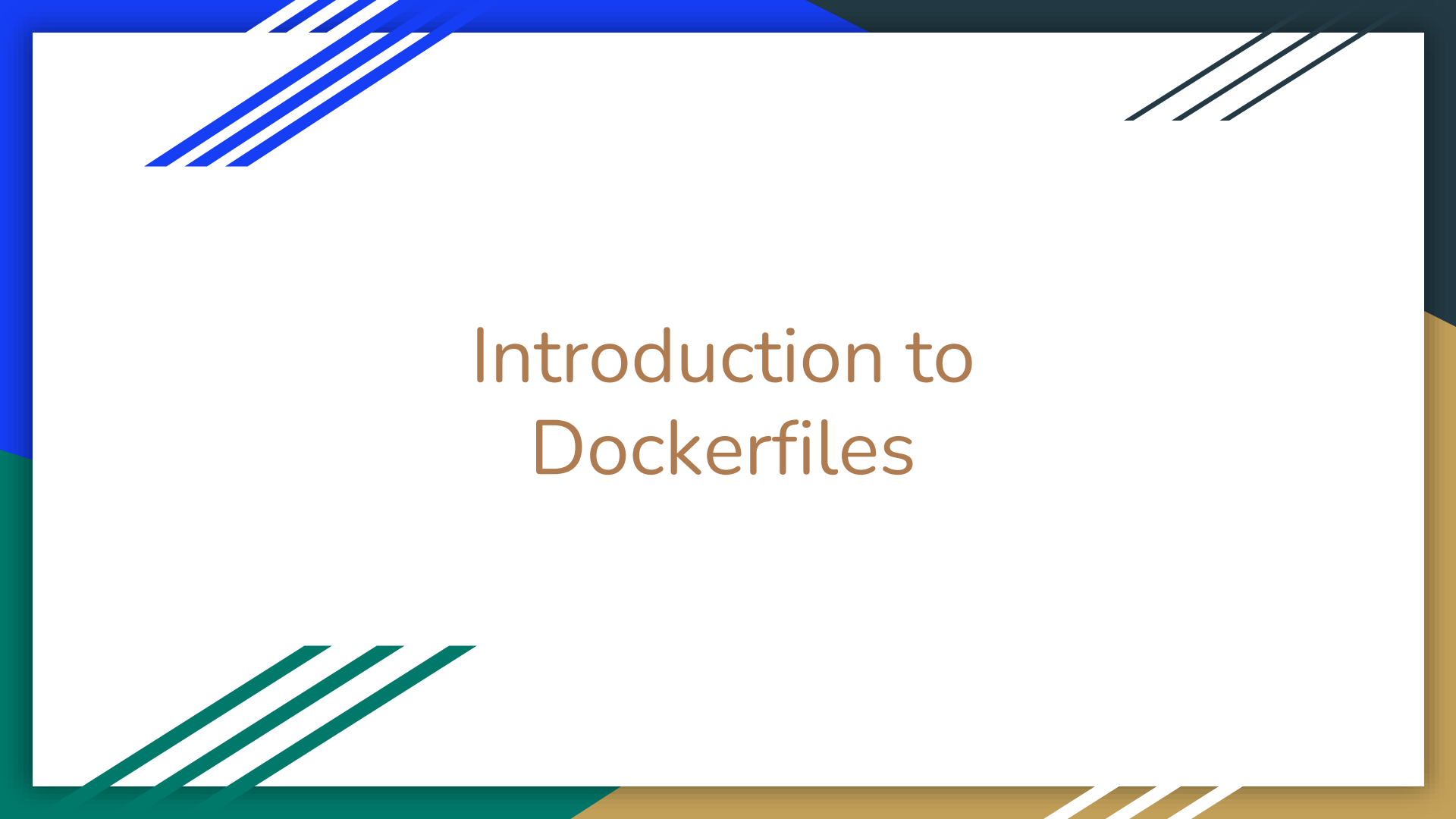
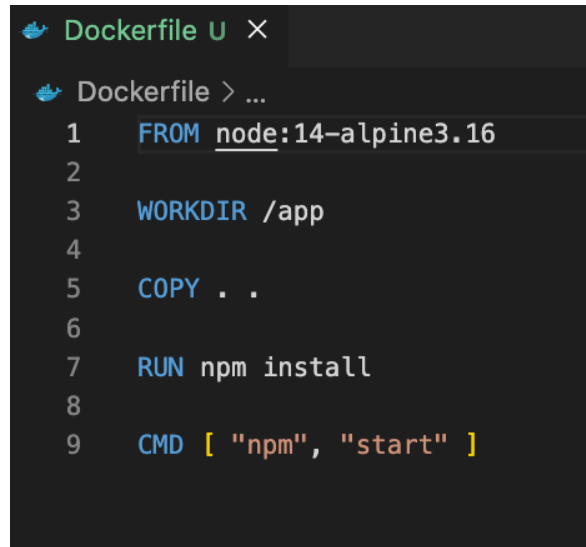




Introduction to Dockerfiles

Understanding Dockerfiles

- A Dockerfile is a text file that contains instructions for automating the process of creating a Docker image. It serves as a blueprint for assembling all the components needed to run an application within a Docker container. Essentially, a Dockerfile outlines the steps required to configure and set up an environment within a container, including installing dependencies, configuring settings, and executing commands.
- The primary purpose of a Dockerfile is to facilitate the reproducibility and portability of applications across different environments. By defining the exact steps needed to build an image, Dockerfiles ensure consistency and reliability in the deployment process. Additionally, Dockerfiles promote the use of infrastructure as code (IaC) principles, enabling developers to version control and manage their application environments alongside their codebase.



```
Dockerfile U X
Dockerfile > ...
1 FROM node:14-alpine3.16
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN npm install
8
9 CMD [ "npm", "start" ]
```

Benefits of Using Dockerfiles

- **Automation:** Dockerfiles automate the process of building Docker images, eliminating the need for manual intervention.
- **Consistency:** With Dockerfiles, developers can ensure consistency in the deployment process by specifying the exact dependencies and settings required for their application.
- **Version Control:** Dockerfiles can be version controlled alongside the application code, enabling developers to track changes and manage updates effectively.
- **Scalability:** Dockerfiles enable scalable application deployments by providing a lightweight and efficient containerization solution.

Key Components of a Dockerfile

- **FROM:** The FROM instruction specifies the base image upon which the new image will be built.
- **RUN:** The RUN instruction executes commands within the Docker image during the build process.
- **CMD:** The CMD instruction defines the default command to be executed when the Docker container starts.
- **COPY/ADD:** The COPY and ADD instructions copy files and directories from the host machine into the Docker image.
- **ENV:** The ENV instruction sets environment variables within the Docker image.
- **EXPOSE:** The EXPOSE instruction exposes specific ports on the Docker container to allow communication with other containers or external services.
- **WORKDIR:** The WORKDIR instruction sets the working directory for subsequent instructions in the Dockerfile.

The FROM Instruction

- The FROM instruction in a Dockerfile sets the groundwork for building Docker images. Essentially, it defines the starting point, dictating the environment within which an application operates. This instruction is vital because it ensures consistency across different development and deployment environments. By specifying a base image, developers encapsulate all necessary dependencies and configurations, streamlining the deployment process. Moreover, version control is facilitated through the FROM instruction, allowing developers to track changes and ensure reproducibility. Security is also enhanced as developers can select trusted base images and regularly update them to patch vulnerabilities. Overall, the FROM instruction forms the cornerstone of Docker image creation, influencing factors like compatibility, security, and maintainability.

The RUN Instruction

- The RUN instruction in Dockerfiles holds the power to execute commands within the Docker image during the build process. This instruction is pivotal for installing packages, configuring settings, and performing setup tasks essential for the application. By leveraging the RUN instruction, developers automate the build process, reducing manual intervention and minimizing errors. Moreover, this instruction allows for the customization of the Docker image, enabling developers to tailor the environment to suit the specific needs of the application. From installing dependencies to executing scripts, the RUN instruction empowers developers to streamline the Docker image creation process, ensuring efficiency and reliability in deployment workflows.

The CMD Instruction

- The CMD instruction in Dockerfiles is a critical element that defines the primary command executed when a container launches. Unlike the RUN instruction, which operates during the build phase, CMD comes into play during runtime, determining the container's initial behavior.
- Essentially, CMD sets the default action of the container, usually representing the main application or service it hosts. This instruction simplifies the deployment process by automating the startup procedure, ensuring that containers kick off with the desired functionality without manual intervention.

The LABEL Instruction

- The LABEL instruction in Dockerfiles serves a crucial role in annotating Docker images with metadata. This metadata provides key information about the image, such as version numbers, maintainer details, and descriptive tags.
- By utilizing the LABEL instruction, developers enhance the manageability and traceability of Docker images throughout their lifecycle. Labels can convey various types of information, including licensing information, project URLs, and release notes, making it easier to understand the purpose and usage of the image.

The EXPOSE Instruction

- The EXPOSE instruction in Dockerfiles plays a vital role in informing Docker about which network ports should be exposed by the container. This instruction does not actually publish the ports; instead, it serves as documentation for developers and operators regarding the intended network connections.
- By specifying ports with the EXPOSE instruction, developers clarify the networking requirements of their containerized applications. This documentation is particularly valuable when collaborating with teams or deploying applications across different environments.
- Although the EXPOSE instruction exposes ports, it doesn't automatically make them accessible from outside the container. To enable external access to exposed ports, additional configuration, such as port mapping during container runtime, is required.

The ENV Instruction

- The ENV instruction in Dockerfiles is instrumental in setting environment variables within the Docker image. Environment variables are dynamic values that can influence the behavior of software applications and services running inside the container.
- By utilizing the ENV instruction, developers can define key parameters such as database connection strings, API endpoints, and application-specific settings directly within the Dockerfile. This simplifies the configuration process and enhances the portability of Docker images across different environments.
- Moreover, environment variables set with the ENV instruction can be accessed by subsequent instructions in the Dockerfile and persist within the container at runtime. This ensures consistency and reproducibility in the behavior of containerized applications across various deployment scenarios.

The ARG Instruction

- The ARG instruction in Dockerfiles allows developers to define build-time variables that can be passed to the Docker build process. These variables provide flexibility and customization options during image creation, enabling developers to parameterize their Dockerfiles and make them more adaptable to different scenarios.
- ARG variables are passed to the Docker build command using the `--build-arg` flag, allowing developers to override their default values at build time. This provides greater control and flexibility, particularly in automated build pipelines or scenarios where different configurations are required for different environments.

ARG vs. ENV Instructions

ARG (Build-time variables):

- Used to pass arguments at build-time.
- Values are accessible only during the build process and not persisted in the final image.
- Useful for providing dynamic values to the Dockerfile, such as version numbers or file paths.
- Syntax: *ARG variable_name=default_value*

ENV (Run-time variables):

- Sets environment variables that are accessible during container runtime.
- Values are persisted in the final image and can be overridden during container instantiation.
- Often used to configure application settings or define runtime behavior.
- Syntax: *ENV variable_name=value*

The ADD Instruction

- The ADD instruction in Dockerfiles facilitates the inclusion of files and directories from the host machine into the Docker image. This instruction is versatile, allowing developers to add a wide range of resources, including application code, configuration files, and static assets.
- By leveraging the ADD instruction, developers streamline the Docker image creation process, ensuring that all necessary components are readily available within the container. This simplifies the deployment process and reduces dependencies on external resources, enhancing the portability and self-sufficiency of Docker images.

The COPY Instruction

- The COPY instruction in Dockerfiles facilitates the transfer of files and directories from the host machine into the Docker image. This instruction is fundamental for incorporating application code, configuration files, and other resources required for the containerized application.
- By utilizing the COPY instruction, developers ensure that all necessary assets are included within the Docker image, streamlining the deployment process and reducing dependencies on external resources. Additionally, COPY offers transparency and control over the file transfer process, enhancing the reproducibility and reliability of Docker image builds.

ADD vs. COPY Instructions

COPY:

- COPY is straightforward and primarily intended for copying local files and directories into the image.
- It offers transparency and simplicity, making it a safer choice for most file transfer tasks.
- COPY does not support URLs or automatically unpack compressed files, focusing solely on copying files from the host machine.

ADD:

- ADD is more versatile and offers additional functionality beyond simple file copying.
- In addition to local files and directories, ADD supports URLs and can automatically unpack compressed files.
- While ADD provides flexibility, it carries potential security risks when used with URLs, as it may inadvertently introduce vulnerabilities if the source is untrusted.

The ENTRYPOINT Instruction

- The ENTRYPOINT instruction in Dockerfiles sets the primary command that will be executed when a container starts. Unlike the CMD instruction, which can be overridden at runtime, ENTRYPOINT defines an immutable starting point for the container.
- This instruction is typically used to specify the main executable or process that the container will run. It provides consistency and predictability in container behavior, ensuring that essential services or applications are always started as intended.
- By leveraging the ENTRYPOINT instruction, developers establish a clear entry point for containerized applications, enhancing their usability and maintainability. This instruction is especially valuable in scenarios where the containerized application requires a specific entry point or initialization sequence.

CMD vs. ENTRYPOINT

CMD:

- CMD sets default command and/or parameters which can be overridden during container runtime.
- It is often used to define the main executable or process that the container runs by default.
- CMD is flexible and allows users to specify alternative commands when running the container.

ENTRYPOINT:

- ENTRYPOINT sets the primary command that will always be executed when a container starts.
- It provides an immutable starting point for the container and cannot be overridden at runtime.
- ENTRYPOINT is typically used to define the main executable or process that the container must run.

The VOLUME Instruction

- The VOLUME instruction in Dockerfiles facilitates the creation of mount points within the container, enabling access to directories on the host machine or other containers. This instruction is essential for managing persistent data and sharing files between the container and its environment.
- By utilizing the VOLUME instruction, developers can specify directories within the container that should be persisted beyond the container's lifecycle. This allows for seamless data sharing and storage between the container and the host machine or other containers in the same Docker network.

The USER Instruction

- The USER instruction in Dockerfiles sets the user context for the container, defining the user account under which the containerized application runs. This instruction is crucial for enhancing security and managing permissions within the container environment.
- By specifying a non-root user with the USER instruction, developers mitigate potential security risks associated with running containers as the root user. This practice follows the principle of least privilege, reducing the impact of security vulnerabilities and unauthorized access to sensitive system resources.

The WORKDIR Instruction

- The WORKDIR instruction in Dockerfiles defines the working directory for the container, specifying the directory path where commands will be executed by default. This instruction simplifies file operations and command execution within the container, enhancing efficiency and readability of Dockerfiles.
- By utilizing the WORKDIR instruction, developers can set a consistent directory structure for their containerized applications, ensuring that relative paths are resolved correctly. This promotes portability and ease of use, as developers can rely on a predefined directory structure regardless of the host environment.

The ONBUILD Instruction

- The ONBUILD instruction in Dockerfiles defines triggers that execute when the image is used as the base for another build. These triggers enable developers to automate actions or configurations that should occur in dependent images, streamlining the Docker image inheritance process.
- When an image with ONBUILD instructions is used as the base for another build, the specified commands are added to the child image's Dockerfile. This allows for the automatic execution of pre-defined actions, such as copying files, installing dependencies, or running scripts, without modifying the original Dockerfile.
- ONBUILD triggers are useful for creating reusable and extensible Docker images that can adapt to different use cases or environments. They provide a way to encapsulate common build steps or configurations, reducing duplication and improving consistency across multiple projects or deployments.

Best Practices for Efficient Images

- **Consolidate Commands:** Combine related commands into single RUN instructions to minimize the number of layers created, reducing image size and build time.
- **Use Multi-stage Builds:** Employ multi-stage builds to separate build dependencies from the final production image, reducing its size by excluding unnecessary build tools and artifacts.
- **Cache Dependencies:** Arrange Dockerfile instructions to maximize cache utilization, benefiting from cached layers and speeding up subsequent builds.
- **Optimize COPY and ADD:** Copy only necessary files into the image, avoiding large directories or unnecessary files, and utilize .dockerignore to exclude irrelevant content.
- **Minimize Image Size:** Remove unnecessary files and dependencies to reduce the image size, opt for lightweight base images like Alpine Linux, and avoid installing unnecessary packages.
- **Cleanup:** Remove temporary files and unused dependencies within the same layer where they are created to minimize the overall image size.
- **Review and Refactor:** Regularly review and refactor Dockerfiles to ensure adherence to best practices, keeping them simple, readable, and maintainable.

Optimizing Dockerfile Caching

- **Immutable Layers:** Prioritize immutable layers to maximize caching efficiency.
- **Order Instructions:** Arrange instructions from least to most changing for better caching.
- **Cache Utilization:** Use cache-friendly instructions like COPY and explicit version tags.
- **Separate Dependencies:** Employ multi-stage builds to isolate dependency installation.
- **Layer Optimization:** Consolidate commands to minimize layer creation.
- **Monitoring:** Regularly monitor build performance to identify optimization areas.

Multi-stage Builds

```
FROM eclipse-temurin:17-jdk-alpine as builder
WORKDIR /opt/app
COPY .mvn/ .mvn
COPY mvnw pom.xml ./
RUN ./mvnw dependency:go-offline
COPY ./src ./src
RUN ./mvnw clean install

FROM eclipse-temurin:17-jre-alpine
WORKDIR /opt/app
COPY --from=builder /opt/app/target/*.jar /opt/app/*.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/opt/app/*.jar"]
```

Multi-stage builds in Docker is a methodology for constructing Docker images that involves dividing the image creation process into multiple stages. Each stage performs specific tasks such as compiling software, running tests, or deploying applications, and can have its own execution context and dependencies.

- **Size Optimization:** By discarding unnecessary artifacts and dependencies in intermediate stages, multi-stage builds result in smaller and more efficient final images.
- **Enhanced Security:** Build-time tools and dependencies are isolated from the final production image, reducing potential attack vectors and enhancing security.
- **Workflow Efficiency:** With multi-stage builds, you can automate complex build processes within a single Dockerfile, streamlining development workflows and ensuring consistency across environments.
- **Faster Builds:** Eliminating the need for intermediate images and optimizing the build process speeds up the overall build time, enabling faster iteration cycles and quicker deployment of applications.

Security Best Practices in Dockerfiles

- In Dockerfiles, security is paramount for safeguarding containerized applications. One fundamental security practice is to avoid running containers with root privileges. Instead, utilizing the `USER` instruction allows specifying a non-root user for container execution, thereby reducing the potential impact of security vulnerabilities. Additionally, relying on official Docker images from trusted sources is crucial, as they undergo rigorous security testing and maintenance. Regularly updating base images and dependencies is essential to patch known vulnerabilities and apply the latest security fixes. Moreover, minimizing the attack surface by removing unnecessary packages and files from images helps mitigate the risk of potential exploits. Implementing the principle of least privilege further strengthens security by limiting container permissions to only what is necessary for the application to function. Enabling Docker Content Trust provides an added layer of security by verifying the integrity and authenticity of Docker images. Lastly, utilizing image scanning tools aids in identifying and addressing security vulnerabilities in Docker images before deployment, ensuring a robust security framework for containerized applications.

Leveraging Official Base Images

- When you choose to use official Docker images as the base for your containers, you're opting for a foundation that's been built and maintained by Docker itself. These images are not only regularly updated and maintained, but they also undergo thorough testing and verification, ensuring their reliability and security.
- One of the key advantages is the trustworthiness that comes with these images. You can rely on them to provide a stable and secure environment for your applications. Additionally, official images adhere to strict quality standards, ensuring consistency across different environments.
- Moreover, official images benefit from extensive community support and documentation. This means that if you encounter any issues or need assistance, you're likely to find help readily available from the vibrant Docker community.

Managing Permissions in Dockerfiles

- **Use USER Instruction:** Specify a non-root user with appropriate permissions using the USER instruction. This helps minimize security risks associated with running containers as the root user.
- **Set Permissions Explicitly:** When copying files into the container using the COPY or ADD instructions, explicitly set permissions using the --chown option to ensure that the files are owned by the appropriate user.
- **Consider Volume Mounts:** When working with volume mounts, ensure that the host directory permissions are properly configured to prevent unauthorized access from within the container.
- **Avoid 777 Permissions:** Avoid using overly permissive file permissions (e.g., 777) as they can pose security risks by granting unnecessary access to files and directories.
- **Follow Principle of Least Privilege:** Apply the principle of least privilege by only granting the necessary permissions for files and directories within the container, minimizing the potential impact of security breaches.

Incorporating Health Checks

- In Dockerfiles, incorporating health checks using the HEALTHCHECK instruction is crucial for maintaining the health and stability of your containers.
- To do this effectively, you specify a command in your Dockerfile that Docker will run periodically to determine the health status of your container. This command could be anything from pinging a specific endpoint to querying a database or executing a custom script, depending on the requirements of your application.
- It's important to configure the frequency at which Docker performs these health checks using the --interval flag. This ensures that your container's health status is monitored regularly. Additionally, you can set a timeout period using the --timeout flag, determining how long Docker will wait for a response before considering the check failed.

Using .dockerignore Files

- When you build a Docker image, Docker collects all the files and directories in the specified build context directory and sends them to the Docker daemon. This directory typically contains the Dockerfile and any other necessary files for building the image.
- However, not all files in the build context are relevant to the image creation process. Some files, such as development dependencies, build artifacts, or sensitive configuration files, may not be needed and can unnecessarily increase the size of the build context.
- This is where the .dockerignore file comes into play. It allows you to specify patterns for files and directories that you want Docker to ignore during the build process. These patterns follow the same syntax as .gitignore files, making it easy to specify which files or directories to exclude based on their names, extensions, or paths.

Understanding Build Context in Docker

- The build context is the set of files and directories that Docker uses when building an image. It includes the Dockerfile and any other files or directories specified in the build command.
- When you initiate a build, Docker sends the entire build context to the Docker daemon. This means that the size of the build context directly affects the duration of the build process.
- A larger build context takes longer to transfer to the Docker daemon and increases the overall build time. Conversely, a smaller build context speeds up the build process by reducing transfer time and improving overall efficiency.
- Therefore, it's essential to carefully manage the contents of the build context, ensuring that only necessary files and directories are included. This is where tools like `.dockerignore` files come in handy, allowing you to exclude irrelevant files and directories from the build context.
- By optimizing the build context, you can minimize build times, improve performance, and enhance the efficiency of your Docker image creation process.

Debugging Docker Builds

- **Review Dockerfile:** Carefully inspect your Dockerfile for syntax errors, missing commands, or dependencies.
- **Check Build Context:** Ensure all necessary files are included in the build context, using `.dockerignore` to exclude irrelevant files.
- **Inspect Logs:** Analyze build logs for error messages or warnings to identify issues.
- **Use `--no-cache`:** Avoid cached layers with `--no-cache` flag to troubleshoot caching-related problems.
- **Debugging Tools:** Utilize Docker commands like `docker inspect` and `docker history` to examine image metadata and layer history.
- **Interactive Builds:** Run Docker in interactive mode (`-i` flag) for real-time debugging during builds.
- **Community Resources:** Seek assistance from the Docker community through forums and documentation for expert advice.

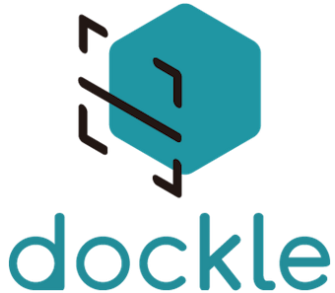
Dynamic Dockerfile Generation

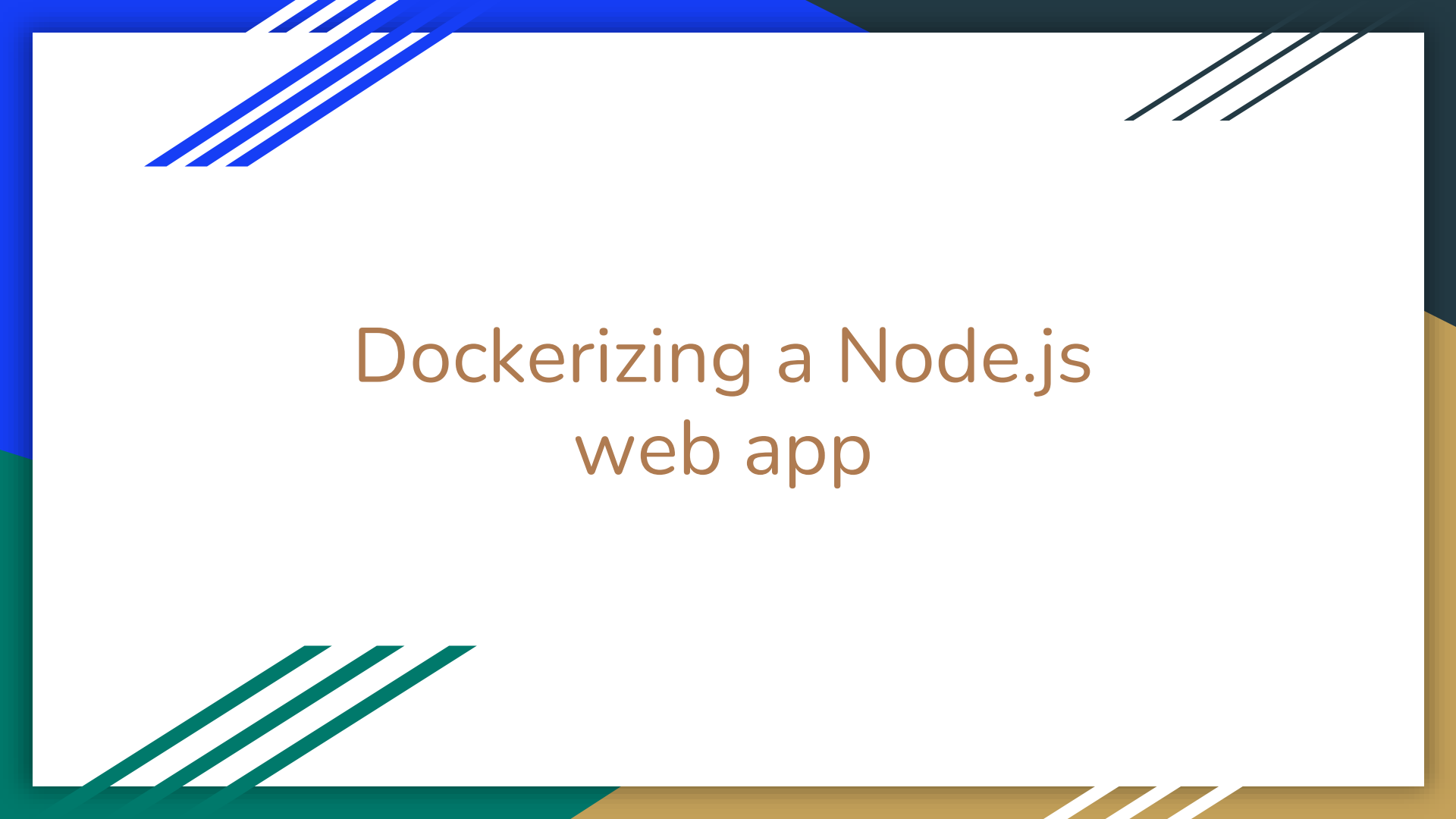
- **Template Engines:** Utilize template engines like Jinja2 or Go templates to generate Dockerfiles dynamically. These engines enable the insertion of variables, conditions, and loops into Dockerfile templates, allowing for customization based on dynamic parameters.
- **Scripting Languages:** Use scripting languages such as Python, Bash, or PowerShell to generate Dockerfiles programmatically. These scripts can read configuration files, environment variables, or other inputs to generate Dockerfile content dynamically.
- **Build-time Arguments:** Leverage build-time arguments (ARG) in Dockerfiles to pass dynamic values during the build process. This allows for parameterization of Dockerfile instructions based on build-time inputs.
- **Multi-stage Builds:** Employ multi-stage builds to generate Dockerfiles dynamically based on different stages of the build process. Each stage can have its own set of instructions and dependencies, allowing for complex and customized Dockerfile generation.
- **External Tools:** Use external tools or libraries specifically designed for Dockerfile generation. These tools may offer advanced features and functionalities for generating Dockerfiles dynamically, simplifying the process and enhancing flexibility.

Dockerfile Linters



- Hadolint: Hadolint is a Dockerfile linter that focuses on best practices and common mistakes.
- Dockerfilelint: Dockerfilelint is a lightweight linter for Dockerfiles that checks for syntax errors and potential issues.
- Trivy: Trivy is primarily a vulnerability scanner for container images, but it also includes Dockerfile linting capabilities.
- Dockle: Dockle is a security and best practices linter for Dockerfiles and container images.
- Linters in IDEs: Some integrated development environments (IDEs) offer built-in support for Dockerfile linting.





Dockerizing a Node.js web app

In the first part of this guide we will create a simple web application in Node.js, then we will build a Docker image for that application, and lastly we will instantiate a container from that image.

Docker allows you to package an application with its environment and all of its dependencies into a "box", called a container. Usually, a container consists of an application running in a stripped-to-basics version of a Linux operating system. An image is the blueprint for a container, a container is a running instance of an image.

Create the Node.js app

First, create a new directory where all the files would live. In this directory create a package.json file that describes your app and its dependencies:

```
{
  "name": "docker_web_app",
  "version": "1.0.0",
  "description": "Node.js on Docker",
  "author": "First Last <first.last@example.com>",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.16.1"
  }
}
```

Create the Node.js app

With your new package.json file, run npm install. If you are using npm version 5 or later, this will generate a package-lock.json file which will be copied to your Docker image.

Then, create a server.js file that defines a web app using the [Express.js](#) framework:

```
'use strict';

const express = require('express');

// Constants
const PORT = 8080;
const HOST = '0.0.0.0';

// App
const app = express();
app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(PORT, HOST);
console.log(`Running on http://${HOST}:${PORT}`);
```

In the next steps, we'll look at how you can run this app inside a Docker container using the official Docker image. First, you'll need to build a Docker image of your app.

Creating a Dockerfile

Create an empty file called Dockerfile:

```
touch Dockerfile
```

Open the Dockerfile in your favorite text editor

The first thing we need to do is define from what image we want to build from. Here we will use the latest LTS (long term support) version 12 of node available from the [Docker Hub](#):

```
FROM node:12
```

Creating a Dockerfile

Next we create a directory to hold the application code inside the image, this will be the working directory for your application:

```
# Create app directory
WORKDIR /usr/src/app
```

This image comes with Node.js and NPM already installed so the next thing we need to do is to install your app dependencies using the npm binary. Please note that if you are using npm version 4 or earlier a package-lock.json file will not be generated.

```
# Install app dependencies
# A wildcard is used to ensure both package.json AND package-lock.json are copied
# where available (npm@5+)
COPY package*.json ./

RUN npm install
# If you are building your code for production
# RUN npm ci --only=production
```

Creating a Dockerfile

Note that, rather than copying the entire working directory, we are only copying the package.json file. This allows us to take advantage of cached Docker layers. Furthermore, the `npm ci` command, specified in the comments, helps provide faster, reliable, reproducible builds for production environments.

To bundle your app's source code inside the Docker image, use the `COPY` instruction:

```
# Bundle app source
COPY . .
```

Your app binds to port 8080 so you'll use the `EXPOSE` instruction to have it mapped by the docker daemon:

```
EXPOSE 8080
```

Creating a Dockerfile

Last but not least, define the command to run your app using CMD which defines your runtime. Here we will use node server.js to start your server:

```
CMD [ "node", "server.js" ]
```

Your Dockerfile should now look like this:

```
FROM node:12

# Create app directory
WORKDIR /usr/src/app

# Install app dependencies
# A wildcard is used to ensure both package.json AND package-lock.json are copied
# where available (npm@5+)
COPY package*.json ./

RUN npm install
# If you are building your code for production
# RUN npm ci --only=production

# Bundle app source
COPY . .

EXPOSE 8080

CMD [ "node", "server.js" ]
```

.dockerignore file

Create a .dockerignore file in the same directory as your Dockerfile with following content:

```
node_modules  
npm-debug.log
```

This will prevent your local modules and debug logs from being copied onto your Docker image and possibly overwriting modules installed within your image.

Building your image

Go to the directory that has your Dockerfile and run the following command to build the Docker image. The -t flag lets you tag your image so it's easier to find later using the docker images command:

```
docker build -t <your username>/node-web-app .
```

Your image will now be listed by Docker:

```
$ docker images
```

```
# Example
```

REPOSITORY	TAG	ID	CREATED
node	12	1934b0b038d1	5 days ago
<your username>/node-web-app	latest	d64d3505b0d2	1 minute ago

Run the image

Running your image with `-d` runs the container in detached mode, leaving the container running in the background. The `-p` flag redirects a public port to a private port inside the container. Run the image you previously built:

```
docker run -p 49160:8080 -d <your username>/node-web-app
```

Print the output of your app:

```
# Get container ID
$ docker ps

# Print app output
$ docker logs <container id>

# Example
Running on http://localhost:8080
```

Run the image

If you need to go inside the container you can use the exec command:

```
# Enter the container  
$ docker exec -it <container id> /bin/bash
```

Test

To test your app, get the port of your app that Docker mapped:

```
$ docker ps
```

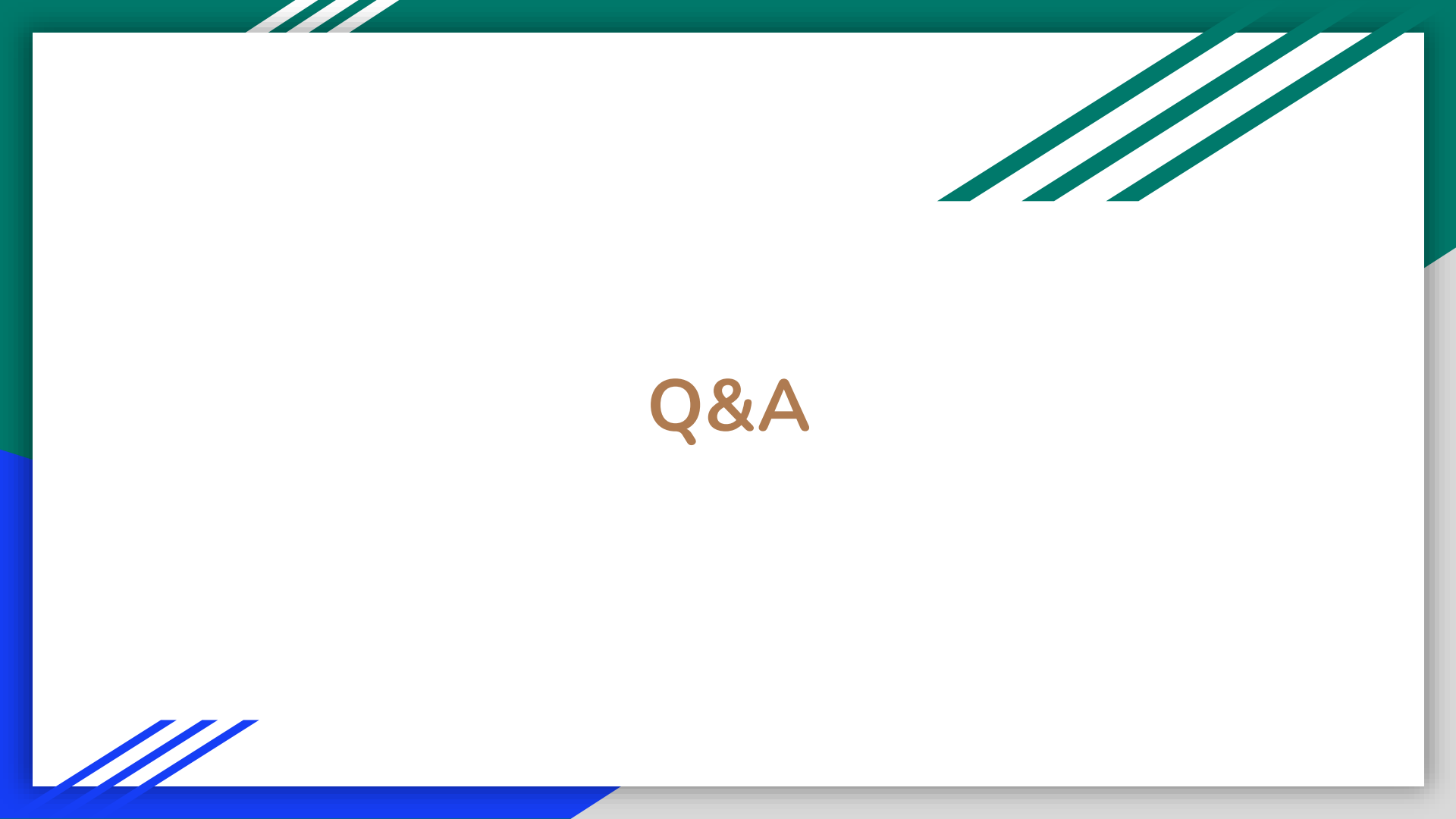
ID	IMAGE	COMMAND	...	PORTS
ecce33b30ebf	<your username>/node-web-app:latest	npm start	...	49160->8080

In the example above, Docker mapped the 8080 port inside of the container to the port 49160 on your machine. Now you can call your app using curl (install if needed via: `sudo apt-get install curl`):

```
$ curl -i localhost:49160
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 12
ETag: W/"c-M6tW0b/Y57lesdjQuHeB1P/qTV0"
Date: Mon, 13 Nov 2017 20:53:59 GMT
Connection: keep-alive
```

Hello world



Q&A